Práctica PDO: Biblioteca del instituto

Presentación

Vas a construir una pequeña aplicación para gestionar la biblioteca del instituto: listar libros, ver su ficha, darlos de alta, prestarlos y devolverlos, y borrarlos. Todo con PDO.

- **Duración:** una sesión (1-2 horas).
- **Objetivo:** practicar las cuatro operaciones CRUD con PDO usando `prepare()` + `execute()`, y `query()` cuando la consulta sea fija.
- **Requisitos previos:** formularios GET/POST, `include`/`require` y funciones en PHP.
- **Material de partida:** el script `biblioteca.sql`, que crea la base de datos `biblioteca` con dos tablas: `libros` (10 libros) y `prestamos` (préstamos que apuntan a algunos libros con clave foránea).

Antes de empezar, importa `biblioteca.sql` en tu MySQL/MariaDB (phpMyAdmin, consola o el cliente que uséis) y comprueba que ves los datos.

Tabla `libros`

Campo	Tipo	Notas
id	INT	Clave primaria, autoincremental
isbn	VARCHAR(17)	Único: no puede repetirse
titulo	VARCHAR(150)	Obligatorio
autor	VARCHAR(100)	Obligatorio
anio	SMALLINT	Año de publicación
genero	VARCHAR(30)	Novela, Ensayo, Poesía, Teatro o Cómic
disponible	TINYINT(1)	1 = en la estantería, 0 = prestado

Normas para toda la práctica

1. Si en la SQL entra cualquier dato variable (de `$_GET`, `$_POST` o de otra variable), se usa `prepare()` + `execute()`. Concatenar variables dentro de la SQL anula el ejercicio.
2. Todo lo que venga de la base de datos y se pinte en HTML pasa por `htmlspecialchars()`.
3. Las operaciones que modifican datos (insertar, actualizar, borrar) se hacen por **POST**, nunca con un enlace GET.
4. El acceso a la base de datos va en funciones dentro de `libros.php`. Las páginas solo llaman a esas funciones y pintan el resultado.

Estructura de ficheros

Entrega una carpeta `biblioteca/` con estos seis ficheros:

Fichero	Qué contiene
`conexion.php`	Función `conectar()` que devuelve un objeto PDO
`libros.php`	Todas las funciones que hablan con la base de datos
`index.php`	Contador de disponibles, filtro por género y listado con botones
`ficha.php`	Ficha de un libro, recibe `?id=` por GET
`nuevo.php`	Formulario de alta y procesado del INSERT
`accion.php`	Recibe por POST las acciones *prestar*, *devolver* y *borrar*

`libros.php` hace `require_once 'conexion.php'`, y cada página hace `require_once 'libros.php'`. Ninguna página escribe SQL directamente.

Ejercicios

Hazlos en orden: cada uno se apoya en el anterior.

1. Conexión

En `conexion.php`, escribe la función `conectar(): PDO`:

- DSN con `charset=utf8mb4` y base de datos `biblioteca`.
- Opciones: `ERRMODE_EXCEPTION`, `FETCH_ASSOC` por defecto y `EMULATE_PREPARES` a `false`.
- Si la conexión falla, captura la `PDOException` y muestra un mensaje genérico (“No se puede conectar con la base de datos”), no el mensaje técnico.

2. Consultas de un único dato

En `libros.php`:

- `contarDisponibles(PDO $pdo): int` devuelve cuántos libros tienen `disponible = 1`. La consulta es fija: usa `query()` y `fetchColumn()`.
- `obtenerLibro(PDO $pdo, int $id): array|false` devuelve la fila de un libro o `false` si no existe. Usa `prepare()`, `execute()` y `fetch()`.

En `ficha.php`, lee `$_GET['id']`, llama a `obtenerLibro()` y muestra todos sus campos, con “Disponible” o “Prestado” en vez de 1/0. Si el id no existe, o no es un número, muestra “Libro no encontrado”.

3. Consultas de varios datos

En `libros.php`, `listarLibros(PDO $pdo, ?string $genero = null): array`:

- Sin género, devuelve todos los libros ordenados por título. Consulta fija: `query()`.
- Con género, devuelve solo los de ese género. Lleva una variable: `prepare()` + `execute()`.

- En los dos casos, `fetchAll()`.

En `index.php`:

1. Muestra arriba “Libros disponibles: N” con `contarDisponibles()`.
2. Añade un formulario GET con un `<select name="genero">` (“Todos” + los cinco géneros) y un botón *Filtrar*. El género elegido debe quedar seleccionado tras filtrar.
3. Pinta una tabla HTML con título, autor, año, género y estado. El título enlaza a `ficha.php?id=...`.
4. Si no hay resultados, muestra “No hay libros de ese género” en vez de una tabla vacía.
5. Añade un enlace “Añadir libro” que lleve a `nuevo.php`.

4. Inserción

En `libros.php`, `insertarLibro(PDO $pdo, array $datos): int` hace el INSERT (con `disponible = 1`) y devuelve el id nuevo con `lastInsertId()`.

En `nuevo.php`, crea un formulario POST con ISBN, título, autor, año y género. Al enviarlo:

- Valida que ISBN, título y autor no estén vacíos, que el año sea un entero entre 1450 y el año actual, y que el género sea uno de los cinco válidos.
- Si hay errores, muéstralos y vuelve a pintar el formulario con lo que el usuario había escrito.
- Si todo va bien, inserta y muestra “Libro añadido con id N” con un enlace al listado.
- Si el ISBN ya existe, MySQL lanza el error 1062. Captúralo y muestra “Ya existe un libro con ese ISBN”.

5. Actualización

En `libros.php`, `cambiarDisponible(PDO $pdo, int $id, int $disponible): int` devuelve las filas afectadas (`rowCount()`). Pon en el WHERE tanto el id como el estado contrario al nuevo, por ejemplo `WHERE id = :id AND disponible = 1` para prestar. Así no se puede prestar un libro que ya está prestado.

En `index.php`, añade a cada fila un formulario POST hacia `accion.php` con un campo oculto `id` y un botón *Prestar* o *Devolver*, según el estado del libro. El botón envía `accion=prestar` o `accion=devolver`.

En `accion.php`, según `$_POST['accion']`, llama a `cambiarDisponible()`. Si `rowCount()` es 0, muestra “No se ha podido cambiar el estado”. Después pon un enlace para volver al listado.

6. Borrado

En `libros.php`, `borrarLibro(PDO $pdo, int $id): int` hace el DELETE y devuelve `rowCount()`.

En `index.php`, añade a cada fila un botón *Borrar* (formulario POST, `accion=borrar`). En `accion.php`:

- Si se ha borrado, “Libro eliminado”.
- Si `rowCount()` es 0, “Ese libro no existe”.

- Los libros que aparecen en la tabla `prestamos` no se pueden borrar: MySQL lanza el error 1451. Captúralo y muestra "No se puede borrar: el libro tiene préstamos registrados". Pruébalo con *Cien años de soledad*.

Preguntas cortas

Respóndelas en un comentario al principio de `index.php` (2-3 líneas cada una).

1. En `listarLibros()` has usado `query()` en un caso y `prepare()` en otro. ¿Por qué? ¿Qué pasaría si escribieras `$pdo->query("SELECT * FROM libros WHERE genero = '$genero'")` y alguien enviara `genero=' OR '1'='1`?
2. ¿Qué devuelve `execute()`? ¿Dónde están entonces las filas del resultado?
3. Pulsa dos veces seguidas *Prestar* en el mismo libro (por ejemplo, abriendo el listado en dos pestañas). ¿Qué devuelve `rowCount()` la segunda vez y por qué?

Ampliación (opcional, para quien termine antes)

- **Ordenar el listado:** añade a `index.php` un parámetro GET `orden` (título, autor o año). Los nombres de columna no se pueden pasar como parámetro de `prepare()`, así que comprúebalos contra una lista blanca con `in_array()`.
- **Buscador:** añade un campo de texto que busque por título o autor con `LIKE`. Pista: el comodín `%` va en el valor que pasas a `execute()`, no en la SQL.
- **Editar:** crea `editar.php`, que cargue los datos de un libro en el formulario y haga el UPDATE de todos sus campos.

Rúbrica de evaluación

La práctica se puntúa sobre 10. La ampliación no suma, pero puede compensar fallos menores.

Criterio	Puntos	Completo	Parcial	No conseguido
1. Conexión	1	`conectar()` con las tres opciones y mensaje genérico de error	Conecta, pero faltan opciones o se muestra el error técnico	No conecta o no está en una función
2. Un único dato	1,5	Contador con `query()`/`fetchColumn()` y ficha con `prepare()`/`fetch()`, que controla el id inexistente	Funciona, pero no controla el id inexistente o usa el método equivocado	No funciona
3. Varios datos	1,5	Listado y filtro correctos, género seleccionado tras filtrar, mensaje si no hay resultados	Listado correcto, pero el filtro falla o falta el mensaje	No funciona
4. Inserción	2	Validación completa, conserva los datos del formulario, muestra	Inserta, pero la validación es	No inserta

Criterio	Puntos	Completo	Parcial	No conseguido
		el id y controla el ISBN duplicado (1062)	incompleta o no controla el 1062	
5. Actualización	1,5	Prestar y devolver por POST, WHERE con el estado y comprobación de `rowCount()`	Funciona, pero sin controlar el estado o sin comprobar `rowCount()`	No funciona o usa GET
6. Borrado	1	Borra por POST, controla el id inexistente y el error 1451	Borra, pero no controla alguno de los dos casos	No funciona o usa GET
7. Seguridad y organización	1	`prepare()` siempre que hay variables, `htmlspecialchars()` en toda la salida y SQL solo en `libros.php`	Falla en uno de los tres puntos	Concatena variables en la SQL
8. Preguntas cortas	0,5	Las tres correctas	Una o dos correctas	Ninguna

Penalización: concatenar una variable dentro de la SQL en cualquier ejercicio deja ese ejercicio a 0.